

5

10

25

9

۷- مخلط: قابلیت اصلی کوپرنشز orchestration است نه مانیپولیشن. با باقی راهکارها هم می توانیم مانیپولیشن داشته باشیم اما نمی توانیم به فوئی کوپرنشز هم نوازسی داشته باشیم

۸- درست. این مفهوم به fast feedback اشاره دارد که در سیستم های AI کاربرد

است. یک AI باعث می شود تست های ضراب شده به جای چند روز در لحظه دقیقه شناسایی شود. البته که AI جایگزین کد ریویو هرگز نیست و صرفاً فرآیند را تسهیل و تسریع می کند.

مورد ۵ و ۸ مقدار ابهام دارد به بسته به زاویه دید هر فرد جوابش متفاوت است.

مثلاً مورد ۵ می تواند درست باشد، اما با این فرض که باقی سرویسها به درستی و فوئی ماک شده باشند.

(سوال ۲)

۱- کش بین لایه‌ها نداریم. باید ترتیب صوری باشد نه لایه‌های باکترین
تجسیرات پایین تر باشند (داخل فصول با شماره کمتر)

۵ مثلاً اول داریم سورس رو کپی می‌کنیم بعد `npm install` زنیغ. باید برعکس باشد

الان با هر تجسیری در کو دوباره باید کتابخانه‌ها نصب بشه

۲- باید ایمیج مستقل از محیط باشد. الان `env` داخل داکر فایل هار د کد شده.

باید `env` ها خارج آر تی فکت باشند و به آن تزریق بشوند

۳- نگ `latest` درست نیست و `bad practice` است. باید از یک ورژن
مشخص استفاده کرد. ممکنه بایک تغییر اساسی در `base image` پروژه خراب بشه.

۴- بهتره از `npm CI` استفاده بشه تا از `package-lock` نصب بشه.

اینجوری دقیقاً همان نسخه نصب میشه و مطمئن تره

۵- چرا `npm run dev` زدیم؟ قوی محیط پرو داکشن نباید از `dev`

استفاده کرد.

۶- از `multi stage` استفاده نشده. بهتره اول بیلد بشه و فرو جسی به محیط
`runtime` کپی بشه تا حجم ایمیج نهایی کمتر بشه.

Parsa

Subject:.....

Date:.....

۷- چرا dackerymore نداره؟ فایل ها و پوشه های اضافی وارد ایسج

می بشوند و حجم را زیاد می کنند.

۱۵- چرا And؟ صحت copy کافی نیست؟

10

15

20

25

Parsa

سوال (۳)

GRPC: این متد بر پایه RPC است و تلاش می کند فراخوانی های شبکه ای را

۵ مانند فراخوانی های local نشان دهد و از protocol buffer استفاده

می کند. (برای انتقال داده)

مزایا: ① به دلیل استفاده از باینری، نسبت به json خیلی سریع تر است.

② type safe است. با استفاده از proto قرارداد بین کلاینت و سرور

مستقل از زبان برنامه نویسی برقرار است.

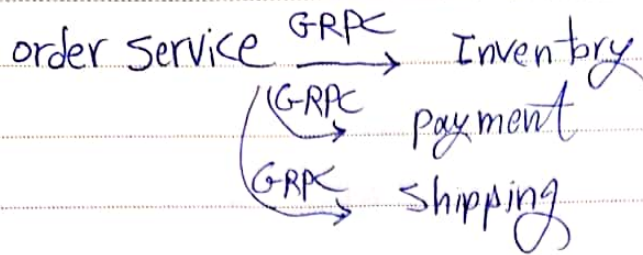
③ از استریم به خوبی پشتیبانی می کند (حتی به صورت دو طرفه)

معایب: ① حذف یک فیلد باعث از کار افتادن consumer های نه در حال

دی سرالانتر از باینری هستند می شود پس باید Expand only باشد پروتو ها چون

② مرورگرها به طور مستقیم از GRPC پشتیبانی نمی کنند.

③ سخت بودن دیباگ. نمی شه payload ها رو راحت خوند چون باینری هستند



مثال واقعی:

ارتباط داخلی میان ~~میکروسروس ها~~ میکروسروس های یک سازمان

5

GraphQL: زبانی برای کوئری کردن است که به کلاسیته ها اجازه می دهد ساختار دیتای

درخواستی شود را تعریف کنند.

مزایا: ① کاهش رفت و برگشت شبکه: می توان nested ها را داخل یک کوئری هاندل کرد.

② تولید فویدگار مستندات از روی اسکیمای

③ یکپارچگی برای کلاسیته های مختلف (وب و موبایل و ...)

15

④ جلوگیری از under fetch و over fetch

معایب: ① پیچیدگی سمت سرور: کوئری های پیچیده فشار زیادی به سرور می آورد
② و باعث مشکلاتی مثل N+1 شود.

20

② کش کردن با http دشوارتر است ③ سربار بیشتری دارد

④ ماهیت Read-oriented دارد و mutation ها کمتر مناسب به نظر

⑤ اصلا مناسب ارتباط بین میکروسروس ها نیست

می بیند 25

Parsa

⑥

Subject:.....

Date:.....

ارتباط با payment باید بلاکینگ و sync باشد یا GPR یا Rest.

ارتباط باقی سرویس ها به صورت Async است. سرویس order پس از برداشت Non-blocking

5 موفق به event را در topic یک Kafka منتشر می کند.

سرویس های دیگر به عنوان Consumer های این topic، پیام event

را دریافت کرده و کار خود را انجام می دهند. پس سرویس ها به جای دریافت

10 دستور مستقیم به event گوش می کنند.

ما از Consistency گذر کرده و به eventual consistency رسیدیم. به جای commit

15 تراکنش به طور کامل، به تراکنش های کوچک commit می شود. اگر سرویس

Inventory پس از برداشت متوجه ناموجود بودن کالا شود باید یک تراکنش ~~برای~~

20 Compensating ایجاد شود. مثلاً بازگشت وجه.

Fault tolerance به شدت افزایش می یابد چون brokerها تقوید تضمین شده دارند.

مأخیز برای کاربر نهایی به شدت کاهش می یابد چون سرویس order دیگر

25 منتظر SMS نیست.

Parsa

Subject:

Date:

مثال واقعی: توی شبکه‌های اجتماعی خیلی کاربرد داره. کلاسیک فیلدهایی که

نیاز دارن رو درخواست می‌کنه. یا مثلاً توی سایت monta هم

کلاسیک فیلترها و پارامترهای سفارشی می‌تواند ست کند و جواب

کوئری مورد نظر را بگیرد.

(سوال ۱)

مشکلات sync blocking: ① ایجاد Cascading Failure: اگر سرویس Downstream

قطع باشد، ریکوئست HTTP ما هم قطعی خود. مثلاً اگر SMS ضرب باشد

۱۵ کل فرآیند ثبت سفارش به مشکل می‌خورد و سرویس order درگیر قطع می‌شود.

② افزایش تاخیر: فرستنده باید منتظر جواب ۱ سرویس باشد پس معیار پریوریتیست.

۲۰ ③ افزایش Coupling: سرویس order نیاز است همه سرویس‌ها را بشناسد. و مستقیم با آن‌ها درگیر شود.

با توجه به این که برداشت باید قطعی باشد و بقیه می‌توانند تاخیر داشته باشند، بهتر است

از Saga استفاده کنیم.

25

Parsa

مزایا: تولید کننده پیام دیگر اهمیتی نمی دهد که مصرف کننده چه کسانی هستند پس

کاهش coupling داریم. مسئله بدون تغییر سروس Analytics اضافه کرد.

5. معایب: این سیستم ~~تولید~~ پیچیدگی ^{داده} رابط‌های بازگشتی است. مثل بازگشت

وجه: همچنین trace کردن در میان سروس‌ها سخت است و نیاز

به correlation ID است.

10

5

۱- Atomicity: یا تمام عملیات ما با موفقیت انجام می شود یا هیچ کدام

انجام نمی شود ما از ناسازگاری داده‌ها با آپدیت‌های ناقص جلوگیری شود.

15

سطح وقوع هم توزیع شده است. چون در سطح ارتباط میان سروس‌ها

رخ داده است. چون الان داده‌ها ناسازگار است و آپدیت ناقص رخ داده است.

۲- Isolation & Consistency: انزولیشن یعنی تراکنش‌های همزمان بدون تداخل

با یکدیگر کار کنند. عدم استفاده از سطوح بالاتر انزول‌سازی (مثل Serializable)

بحث بروز مشکل ریزو تکرار می شود (در فرم‌های همزمان) همچنین

25 منشی شدن موقوعی نشان دهنده نقص سرویس دیتابیس و از بین رفتن ^{موقوع} consistency

سطح وقوع: مثالی: دیتابیس Inventory 9 رخ داده است. شده است.

۳- Durability: این ویژگی تضمین می‌کند که پس از Commit شدن یک تراکنش

داده‌ها حتی در صورت کرش کردن یا قطعی برق حفظ شوند. از بین رفتن داده

۵ بعد از استارت نشان می‌دهد که WAL نداریم. سطح وقوع محلی است

چون این ویژگی همواره باید در سطح دیتابیس per-service باشد و تضمین شود.

پس این نقض مربوط به خود سرویس payment است.

10

۴- Atomicity + Consistency: در صورت شکست یک تراکنش باید تراکنش‌های وابسته برای

تمام مراحل موفق قبلی اجرا شود تا گرهایی که انجام شده بخشی شوند و

۱۵ سیستم به حالت پیش از شروع Saga برگردد. اجرا نشدن این تراکنش‌ها

نقض این ویژگی است و سیستم نیکو می‌ماند. سطح وقوع توزیع شده

است. این مشکل در جریان کاری توزیع شده بین سرویس‌ها رخ داده.

20

چون طراحی فرآیند Rollback به درستی در سطح معماری تمامی مراحل را پوشش

ندارد.

25

Parsa

Subject:

Date:

Isolation - 5: الگوهای مانند Saga و Two-Phase Commit را در سطح توزیع شده فراهم
+ Consistency

نمی کنند و در نتیجه وضعیت کلی مابین در طول فرآیند قابل مشاهده هستند.

5. در این حالت ثبات قوی فدای در دسترس بودن و مقیاس پذیری شده است.

سطح وقوع توزیع شده است. هر سرویس به صورت محلی تراکنش خود را

به درستی و به شکل انزوله کامل فرآیند ثبت تا زود در سطح

توزیع شده انزوله نیست و باعث ایجاد eventual consistency در یک بازه زمانی کوتاه

شده است.

25 سوال 6

نحوه تشخیص:

مشترکها: ابتدا به مشترکهای سطح هسته مثل CPU، RAM و I/O

نگاه می کنیم تا مطمئن شویم مشکل مصرف منابع نیست یا VM خراب نیست.

20

با استفاده از دسته بندی مشترکها بر اساس نگه های متادیتا، مشترکها را بر حسب شماره

نسخه دسته بندی می کنیم. اگر خطا یا مصرف منابع دقیقاً از زمان دیپلوی وجود مشکل

25 از آخرین نسخه است

Parsa

دریابی توزیع شده: بدون داشبورد فقط timeout را در چند سرویس وابسته نشان می دهد

با استفاده از tracing مسیر دقیق درخواست ها در میان سرویس ها را مشاهده می کنیم

Trace ها به ما نشان می دهد که دقیقاً کجا زمان زیادی مصرف می شود

اگر trace در سرویس اصلی متوقف شود مشکل از خود سرویس است اما اگر نشان دهد که سرویس اصلی زمان زیادی را در یک span مانده است، مشکل از وابستگی ها

10

تک ها: باید شناسه همبستگی لاگ های timeout شده را از سیستم tracing برداریم

همین در سیستم تجزیه لاگ، تمام لاگ های مربوط به آن شناسه را جستجو می کنیم

تا دقیقاً متوجه شویم درخواست در سرویس اصلی در چه نقطه ای و با چه فطالی

متوقف شده است

بیش نیازها برای کاهش MTTR:

20

داده های مورد نیاز: ① شناسه همبستگی از همان نقطه اول تولید و در تمامی درخواست ها و در تمامی لاگ ها درج شود

② ثبت متریک های سفارشی و متصل کردن Instance از سرویس به هاست مربوط

③ ثبت زمان های پاسخ دهی برای تمامی رابط های سرویس ها

Parsa

داشودها: داشوردهای دارای high cardinality که اجازه دهد داده‌ها را بر اساس

تگ‌های مختلف (نام سرویس/هاست/شناسه/نصف بیلد/...) فیلتر و ترکیب کنیم

5 تا بتوانیم به سوالاتی که از قبل پیش بینی نکرده بودیم پاسخ دهیم.

نمایش بصری Trace ها برای ردیابی سریع محل گیر کردن درخواست‌ها در

زنجیره میکروسروس ها

10

طراحی هشدارها و مانیتورینگ:

- مانیتورینگ معنایی: هشدارهایی که صرفاً روی معیارهای سطح پایین مثل ریسورس‌ها

15 باشند. بلکه باید بر اساس ~~تاریخچه~~ ~~تاریخچه~~ ~~تاریخچه~~ نتایج واقعی خرابی و کار تنظیم شوند.

- پرهیز از هشدارهای اضافی: در طراحی هشدارها باید همواره پرسید که آیا این مشکل

آن قدر مهم است که یک نفر را ساعت ۳ صبح بیدار کند؟ تا از خستگی ناشی از Alert ها

20

جلوگیری شود.

استفاده از SLI و Error budget تا تیم‌ها بدانند چه میزان قطعی یا

25 تاخیر غیرطبیعی است و نیاز به مداخله فوری دارد؟

Parsa

سوال ۷) برای جلوگیری از هدر رفتن زمان دریافت بازخورد سریع

تست ها و بررسی ها باید به صورت ~~staging~~ مرحله ای اجرا شود.

۱) تست های سریع (تست های واحد و lint و ~~type check~~)

اجرای شود و دارای fast fail هست.

۲) بررسی قرارداد: برای جلوگیری از انتشار نسخه ناسازگار فایل های اسکیم

(openAPI و proto) با نسخه قبلی مقایسه می شوند و در صورت وجود

تغییرات مقرب متوقف می شود.

۳) ساخت آرتیفکت: داکر ایجی ساخته شده و تگ sha می خورد

۴) تست های Integration اجرا می شوند و آرتیفکت ساخته شده در محیط

انزوا تست C2C می شود.

۵) استقرار pre-production: آرتیفکت + env های آن روی محیط staging

منتشر می شود.

⑥ استقرار عملیاتی: استقرار با استفاده از روش‌های zero downtime مانند

Rolling update یا blue/green انجام می‌شود.

تغییرات در API ها همواره باید به صورت Expansion باشد

فیلدهای جدید optional و فیلدهای قبلی هم همچنان باشند.

10 اگر تغییرات نام سازگار الزامی بود باید از استراتژی شبیه سازی رابط قدیمی

استفاده کرد. در این حالت میکروسرویس هر دو حالت ۱۶۱ و ۱۶۲ را

برای API ارائه می‌کند. این باعث می‌شود که باقی سرویس‌ها

15

مجبور به آپدیت هم‌زمان نباشند و با سرعت خودشان به نسخه جدید

مهاجرت کنند. وقتی ترافیک API قبلی به صفر رسید حذف می‌شود.

20 نسخه‌بندی می‌تواند با http path یا header Param انجام شود.

برای مدیریت migration ها می‌توان هر سرویس دیتابیس خود را

با تغییرات دیتابیس هم مثل API ها Expand-only باشد تا به راحتی

25

Parsa

بسته Rollback امن انجام داد و Rolling update کرد.

در Rolling update هم زمان نسخه قدیم و جدید بالا است پس نباید ستون ها

5 یا حداقل قدیمی را حذف کرد اگر بیش از یک Pod داریم. مهاجرت های

دارای حذف و تغییر باید در دو مرحله انجام شود.

برای Rollback امن از blue Green استفاده می کنیم. با لود بالا ترافیک

10

را به نسخه قدیم می بریم و نسخه جدید را در کنار آن با ترافیک کمتر قرار

می دهیم و در صورت پایداری ترافیک را کم کم به سمت آن می بریم.

15 در صورت هر گونه مشکل ترافیک به سمت blue می رود و ترافیک Green

صفر می شود. همچنین می توانیم از Feature flag استفاده کنیم.

کدهای جدید به صورت خاموش در production مستقر می شود. این الگو

20

اجازه می دهد قابلیت های جدید به صورت درصدی برای کاربران فعال

شوند. در صورت بروز خطا فوراً بدون نیاز به دیپلوی مجدد

Rollback می شود. مفودار باید لاین CI/CD :

Parsa

(سوال ۸)

نیاز مندی های عملکردی و سرویس ها:

5 ① سرویس سفارش: پوشش سفارش تحدا و بگیری وضعیت تحدا

عملکرد: باید state machine حرفه عملکرد سفارش را مدیریت می کند

هر تغییر وضعیت یک event تولید می کند و publish می شود

10

2 ② سرویس رزرو: پوشش دهنده رزرو صندلی برای افرای رزده

عملکرد: دیباچیس این سرویس برای جلوگیری از رزرو های هم زمان، سطح

15 اینزول سازی تراکنش بالایی خواهد داشت

3 ③ سرویس وفاداری و مارکتینگ: پوشش دهنده باشگاه مشتریان و صدور کوپن تخفیف

عملکرد: این سرویس نیازی به دریافت درخواست مستقیم همکار از سرویس سفارش

20

ندارد. بلکه رویدادهای سفارش تکمیل شده از Kafka گوش می دهد و امتیازات را

الحاظ می کند

25

Parsa

④ سروس بلاگ و محتوا: پوشش بلاگ

حکمر: یک سروس به شرح read heavy با پیاده سازی مکانیزمهای کشنده

5 اخبار بنده و رستوران را نمایش می دهد

⑤ سروس پشتیبانی: پوشش دهنده پشتیبانی مشتریان در تمام امور

⑥ سروس HR و شعب: پوشش دهنده مدیریت شعبها و کارمندان

⑦ سروس تحلیل و گزارشات: پوشش دهنده گزارش گیری مالی و آمار مشتریان

حکمر: این سروس در جریان ارتباطات همزمان نخالتی ندارد، بلکه تمامی رویه های

15 سیستم را از دسترس پارسا در امان نگاه داشته و در دسترس تحلیل خود جمع آورده

می کند.

نیازمندی های NF:

① مقیاس پذیری: با ورود ناگهانی 1M مشتری نمی توان کل سیستم را یکجا مقیاس دهی

کرد. معماری میکروسروس اجازه می دهد سروس هایی که تحت فشار شدید هستند

25 را به صورت مستقل از سایر بخش ها اسکال کنیم. این کار با کاستریزاسی

Parsa

و استقرار روی کلاستر ①8 کوپرنیتز به راحتی مدیریت می شود.

۲) تاب آوری و تحمل خطا: استفاده از Kafka باعث می شود تا اگر ~~یک~~

سرویس به دلیل حجم بالای ترافیک دچار قطعی شود، پیام های مربوط به ثبت

۵ سفارش در صف منتظر بمانند و دیتایی از دست نرود.

۳) پایداری داده و مدیریت تراکشن های توزیع شده: هر سرویس دیتای مستقل خود

را دارد تا از وابستگی جلوگیری شود. برای فرآیندهایی که نیازمند درگیر شدن

10

ملیند سرویس هستند مثل ثبت سفارش و کسر همزمان کوپن به جای تراکشن های

فصل گشته توزیع شده نه عملکرد را قلع می کنند، از الگوی Saga با قابلیت

۱۵ تراکشن های جبرانی بهره می بریم.

۴) ارزیابی و دیباگینگ: در سیستمی با ۲۸ کاربر، یافتن خطاهای پنهان

در شبکه بدون ابزار خیر ممکن است. این سیستم نیازمند پیاده سازی ردیابی

20

توزیع شده است و تریق Correlation ID در تگ درخواست ها

ولاگ ها است تا بتوان مسیر یک درخواست را در طول تمام سرویس ها

۲۵ مگیری کرد. در سطوح پایین تر هم می تون متد های پروتکستوس را برای مانیتورینگ

Parsa

۱۹) پیاده سازی کرد

Subject:

Date:

External clients $\longleftrightarrow$ Gateway : نوع ارتباطات :
Rest

Gateway $\longleftrightarrow$ Services
GRPC

ارتباطهای ~~میکروسرویس~~ : میان سرویسها: با Saga مدیریت می شود.

در صورت بروز خطای تجاری رویدادهای شکست منتشر شده و تراکنشهای

جبران برای لغو مراحل قبلی اجرا می شوند.

(شودارهای mermaid به کمک AI رسم شده اند)

